

✓ Kolkata Air Quality Forecasting — Enhanced Capstone II

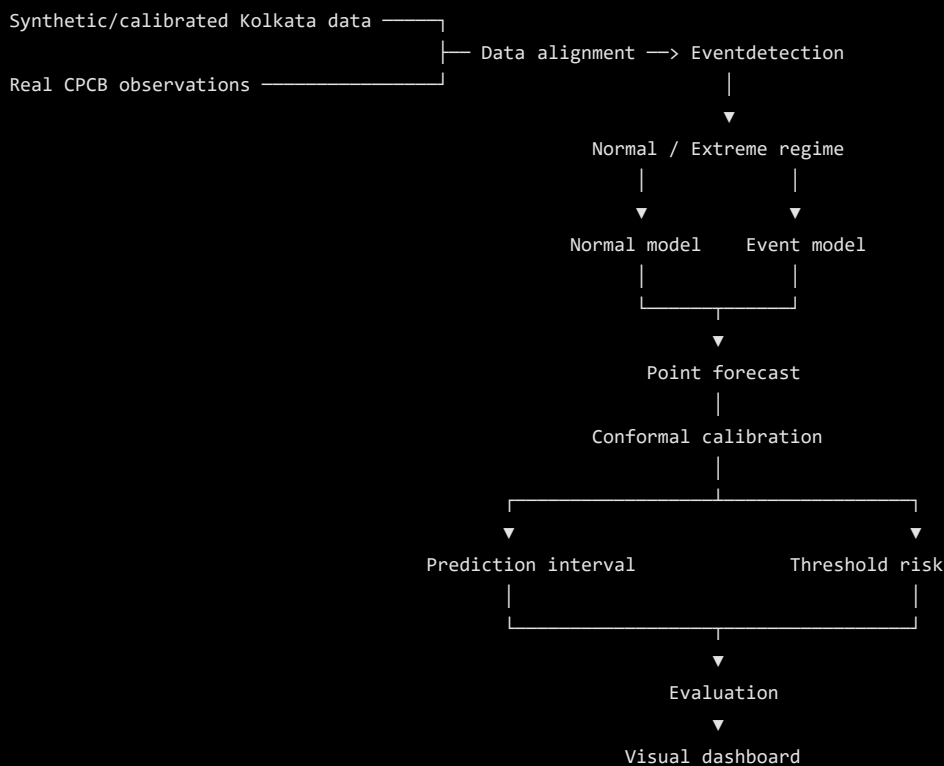
Supervisor-enhancement version

This notebook extends the original SARIMA / Prophet / XGBoost / LSTM study with the three requested research components:

1. **Real sensor-data validation** using CPCB observations against the synthetic/calibrated Kolkata benchmark.
2. **Event-aware adaptive forecasting** using separate normal-condition and extreme-event models.
3. **Uncertainty-aware forecasting** using conformal prediction, prediction intervals, and pollution-threshold exceedance risk.

Important: CPCB files are external inputs. The notebook does not fabricate real sensor observations. If a CPCB CSV is not available locally, the CPCB sections remain runnable in a clearly labelled "data required" state.

✓ Research architecture



```

# =====
# 0. CONFIGURATION
# =====

from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.ensemble import RandomForestClassifier, GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
TARGET = "pm25"
HORIZON = 1
ALPHA = 0.10          # 90% conformal interval
THRESHOLD = 250.0      # configurable PM2.5 threshold; change to your study's adopted standard
  
```

```
DATA_DIR = Path("./data")
SYNTHETIC_PATH = DATA_DIR / "air_quality.csv"
CPCB_PATH = DATA_DIR / "cpcb_kolkata.csv"

np.random.seed(RANDOM_STATE)

print("Synthetic dataset:", SYNTHETIC_PATH)
print("CPCB dataset:", CPCB_PATH)
print("Target:", TARGET)
print("Conformal coverage target:", f"{1-ALPHA:.0%}")
```

```
Synthetic dataset: data\air_quality.csv
CPCB dataset: data\cpcb_kolkata.csv
Target: pm25
Conformal coverage target: 90%
```

```
# =====
# OPTIONAL ENVIRONMENT SETUP
# =====
# Run only if packages are missing in your environment.
#
# %pip install pandas numpy matplotlib scikit-learn xgboost statsmodels prophet tensorflow
#
# CPCB data must be obtained from an appropriate CPCB source/export and placed at:
# ./data/cpcb_kolkata.csv
```

```
# =====
# 1. DATA LOADING UTILITIES
# =====
def find_column(df, candidates):
    lower = {str(c).lower().strip(): c for c in df.columns}
    for candidate in candidates:
        if candidate.lower() in lower:
            return lower[candidate.lower()]
    return None

def load_air_quality_csv(path):
    if not Path(path).exists():
        raise FileNotFoundError(
            f"File not found: {path}. Place the original synthetic/calibrated CSV there."
        )
    df = pd.read_csv(path)
    dt_col = find_column(df, ["datetime", "date", "timestamp", "time"])
    if dt_col is None:
        raise ValueError("No datetime-like column found.")
    df[dt_col] = pd.to_datetime(df[dt_col], errors="coerce")
    df = df.dropna(subset=[dt_col]).sort_values(dt_col).set_index(dt_col)
    df.columns = [str(c).strip().lower().replace(" ", "_") for c in df.columns]
    return df

def load_cpcb_csv(path):
    if not Path(path).exists():
        return None
    df = pd.read_csv(path)
    dt_col = find_column(df, ["datetime", "date", "timestamp", "time"])
    if dt_col is None:
        raise ValueError("CPCB file needs a datetime/date/timestamp/time column.")
    df[dt_col] = pd.to_datetime(df[dt_col], errors="coerce")
    df = df.dropna(subset=[dt_col]).sort_values(dt_col)
    df.columns = [str(c).strip().lower().replace(" ", "_") for c in df.columns]
    return df

synthetic = load_air_quality_csv(SYNTHETIC_PATH)
print(synthetic.shape)
display(synthetic.head())
```

(87672, 10)

	pm25	pm10	no2	co	so2	o3	temp	rh	wind	rain
datetime										
2015-01-01 00:00:00	150.849670	257.310990	32.664554	1.204211	4.816183	1.364593	17.926895	42.058440	0.1	1.085120
2015-01-01 01:00:00	132.370120	254.495468	31.878915	0.860830	1.937702	1.000000	16.655569	37.960572	0.1	0.000000
2015-01-01 02:00:00	157.994518	256.419390	28.320158	0.074682	4.197653	2.516940	17.283748	37.412200	0.1	0.000000
2015-01-01 03:00:00	173.358215	269.406690	33.000651	2.563948	7.383085	1.000000	34.383131	17.713272	0.1	0.393723
2015-01-01 04:00:00	201.753827	335.956343	30.997890	1.326620	2.140768	1.000000	18.198381	39.903303	0.1	0.000000

```
# =====
# 2. BASIC QUALITY CHECKS
# =====

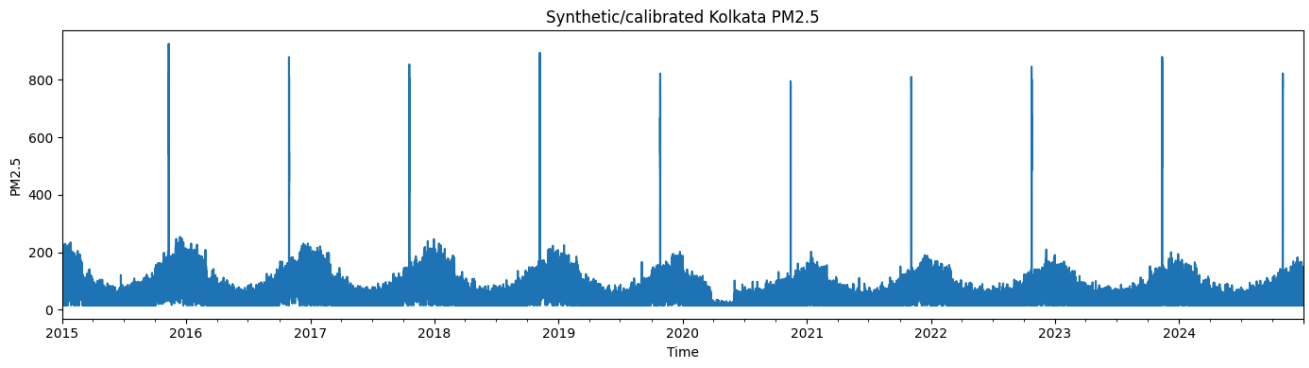
print("Shape:", synthetic.shape)
display(synthetic.isna().sum().to_frame("missing_values").T)
display(synthetic.describe().T)

if TARGET not in synthetic.columns:
    raise ValueError(f'{TARGET!r} not found. Available columns: {list(synthetic.columns)}')

fig, ax = plt.subplots(figsize=(14, 4))
synthetic[TARGET].plot(ax=ax)
ax.set_title("Synthetic/calibrated Kolkata PM2.5")
ax.set_ylabel("PM2.5")
ax.set_xlabel("Time")
plt.tight_layout()
plt.show()
```

Shape: (87672, 10)

	pm25	pm10	no2	co	so2	o3	temp	rh	wind	rain
missing_values	0	0	0	0	0	0	0	0	0	0
count										
mean										
std										
min										
25%										
50%										
75%										
max										
pm25	87672.0	57.828326	46.538527	15.00	20.735777	47.379220	81.719948	926.149477		
pm10	87672.0	98.981629	71.486695	30.00	39.712592	77.363812	140.916957	989.887003		
no2	87672.0	20.380443	12.095623	5.00	11.487912	18.405700	26.568068	84.928413		
co	87672.0	0.999363	1.060623	0.05	0.050000	0.667111	1.665556	6.742515		
so2	87672.0	4.093981	2.791388	0.05	2.112934	3.789012	5.617663	38.134694		
o3	87672.0	12.365964	17.592039	1.00	1.000000	4.964779	12.306806	112.169819		
temp	87672.0	26.967928	9.144992	10.00	20.158294	26.484439	33.506717	45.000000		
rh	87672.0	59.644105	23.614732	5.00	39.763042	58.792463	78.581882	100.000000		
wind	87672.0	1.566229	1.613901	0.10	0.100000	1.168418	2.546975	29.946471		
rain	87672.0	14.651464	50.675309	0.00	0.000000	0.273485	1.914113	351.872020		



3. Event detection and event labels

The original proposal uses STL/S-H-ESD style anomaly detection and a known event calendar. Here we make the event label usable by the adaptive forecasting stage.

The detector below is intentionally modular. If you already have your validated STL/S-H-ESD detector from the original notebook, replace `detect_events()` with that implementation and keep the downstream interface (`event_flag`) unchanged.

```
# =====
# 3. EVENT DETECTION
# =====

def detect_events(df, target=TARGET, window=24, z_threshold=3.0):
    out = df.copy()
    s = out[target].astype(float)
    rolling_med = s.rolling(window, center=True, min_periods=max(3, window//3)).median()
    rolling_std = s.rolling(window, center=True, min_periods=max(3, window//3)).std()
    score = (s - rolling_med).abs() / rolling_std.replace(0, np.nan)
    out["event_score"] = score.fillna(0)
    out["event_flag"] = (out["event_score"] >= z_threshold).astype(int)
    # Group adjacent anomalous hours into event windows and bridge short gaps.
    flags = out["event_flag"].copy()
    gaps = flags.eq(0) & flags.shift(1).eq(1)
    out["event_flag"] = flags
    return out

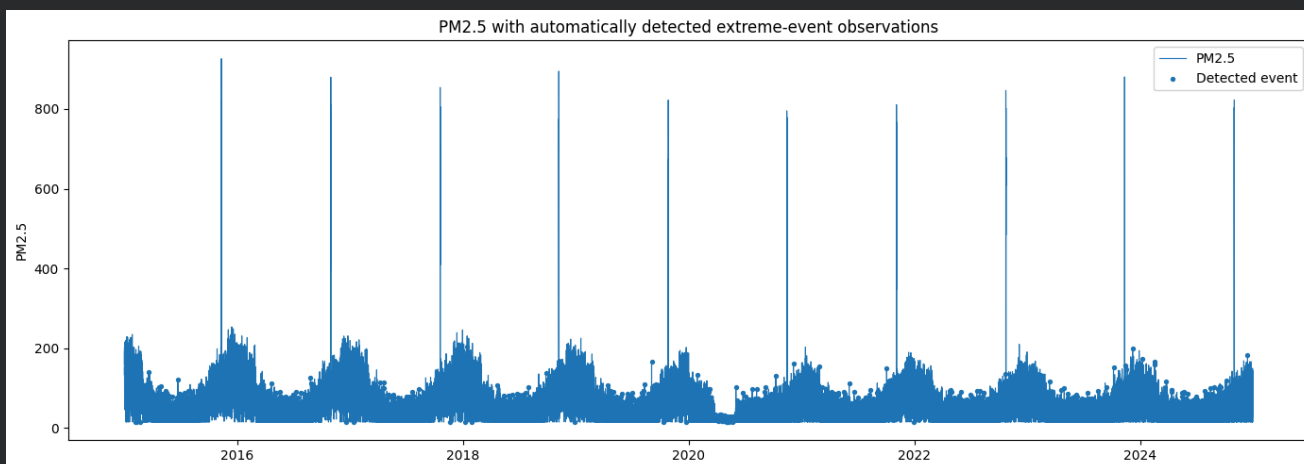
synthetic_events = detect_events(synthetic)

print("Detected event hours:", int(synthetic_events["event_flag"].sum()))
display(synthetic_events[["event_score", "event_flag"]].describe())

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(synthetic_events.index, synthetic_events[TARGET], linewidth=0.8, label="PM2.5")
event_points = synthetic_events[synthetic_events["event_flag"] == 1]
ax.scatter(event_points.index, event_points[TARGET], s=8, label="Detected event")
ax.set_title("PM2.5 with automatically detected extreme-event observations")
ax.set_ylabel("PM2.5")
ax.legend()
plt.tight_layout()
plt.show()
```

Detected event hours: 849

	event_score	event_flag
count	87672.000000	87672.000000
mean	0.751526	0.009684
std	0.694449	0.097929
min	0.000000	0.000000
25%	0.191321	0.000000
50%	0.582340	0.000000
75%	1.122141	0.000000
max	4.903997	1.000000



4. Feature engineering

The forecasting models use lag, rolling, calendar and event-regime features. The same feature generator is used for training and prediction to reduce leakage and make the adaptive model reproducible.

```
# =====
# 4. FEATURE ENGINEERING
# =====

def make_features(df, target=TARGET):
    x = df.copy()
    idx = x.index

    x["hour"] = idx.hour
    x["dayofweek"] = idx.dayofweek
    x["month"] = idx.month
    x["dayofyear"] = idx.dayofyear
    x["is_weekend"] = (idx.dayofweek >= 5).astype(int)

    for lag in [1, 2, 3, 6, 12, 24, 48, 72, 168]:
        x[f"lag_{lag}"] = x[target].shift(lag)

    for window in [3, 6, 12, 24, 72, 168]:
        x[f"roll_mean_{window}"] = x[target].shift(1).rolling(window).mean()
        x[f"roll_std_{window}"] = x[target].shift(1).rolling(window).std()

    x["target_next"] = x[target].shift(-HORIZON)
    return x.dropna()

features = make_features(synthetic_events)
print(features.shape)
display(features.head())
```

(87503, 39)

	pm25	pm10	no2	co	so2	o3	temp	rh	wind	rain	...	roll_std_6	ro
datetime													
2015-01-08 00:00:00	139.864578	258.546585	18.718797	0.050000	3.388007	9.053367	17.169020	37.323693	0.1	0.000000	...	33.500542	
2015-01-08 01:00:00	136.360550	256.782369	23.744791	0.050000	6.189093	14.092747	21.774862	29.637475	0.1	0.359164	...	21.396714	
2015-01-08 02:00:00	164.040555	260.699894	33.183281	0.890238	5.499286	1.000000	12.103603	49.459650	0.1	0.000000	...	9.385320	
2015-01-08 03:00:00	146.149330	257.015440	18.135177	0.957182	4.378363	2.503878	22.334502	34.323944	0.1	0.000000	...	11.077326	
2015-01-08 04:00:00	187.556171	334.774749	27.320891	0.050000	3.588580	7.686394	19.422005	37.906680	0.1	3.676368	...	10.894729	

5 rows × 39 columns

5. Time-based train / validation / test split

No random train/test split is used for the forecasting experiments. The chronological order is preserved.

```
# =====
# 5. CHRONOLOGICAL SPLIT
# =====
def chronological_split(df, train_frac=0.70, val_frac=0.15):
    n = len(df)
    i = int(n * train_frac)
    j = int(n * (train_frac + val_frac))
    return df.iloc[:i].copy(), df.iloc[i:j].copy(), df.iloc[j:].copy()

train, val, test = chronological_split(features)

print("Train:", train.index.min(), "→", train.index.max(), len(train))
print("Validation:", val.index.min(), "→", val.index.max(), len(val))
print("Test:", test.index.min(), "→", test.index.max(), len(test))

Train: 2015-01-08 00:00:00 → 2022-01-03 03:00:00 61252
Validation: 2022-01-03 04:00:00 → 2023-07-04 00:00:00 13125
Test: 2023-07-04 01:00:00 → 2024-12-31 22:00:00 13126
```

6. XGBoost benchmark

XGBoost is used here as the main operational model because its lag/rolling feature structure makes it straightforward to extend into a regime-adaptive system.

If your original notebook already contains tuned XGBoost results, retain those results and use this section as the standardized interface for the enhanced experiments.

```
# =====
# 6. XGBOOST MODEL
# =====
try:
    from xgboost import XGBRegressor
    HAVE_XGB = True
except Exception:
    HAVE_XGB = False
    print("xgboost is not installed; using GradientBoostingRegressor fallback.")

EXCLUDE = {"target_next", TARGET}
feature_cols = [c for c in train.columns if c not in EXCLUDE]
```

```
def make_regressor():
    if HAVE_XGB:
        return XGBRegressor(
            n_estimators=400,
            max_depth=6,
            learning_rate=0.05,
            subsample=0.85,
            colsample_bytree=0.85,
            objective="reg:squarederror",
            random_state=RANDOM_STATE,
            n_jobs=4
        )
    return GradientBoostingRegressor(
        n_estimators=300,
        learning_rate=0.04,
        max_depth=3,
        random_state=RANDOM_STATE
    )

model = make_regressor()
model.fit(train[feature_cols], train["target_next"])

test_pred = model.predict(test[feature_cols])

def regression_metrics(y, pred):
    return {
        "MAE": mean_absolute_error(y, pred),
        "RMSE": mean_squared_error(y, pred) ** 0.5,
        "MAPE_%": np.mean(np.abs((y - pred) / np.maximum(np.abs(y), 1e-6))) * 100
    }

baseline_metrics = regression_metrics(test["target_next"], test_pred)
pd.DataFrame([baseline_metrics], index=["XGBoost baseline"])
```

	MAE	RMSE	MAPE_%
XGBoost baseline	16.1182	23.188218	46.038104

```
# =====
# 7. NORMAL VS EVENT EVALUATION
# =====

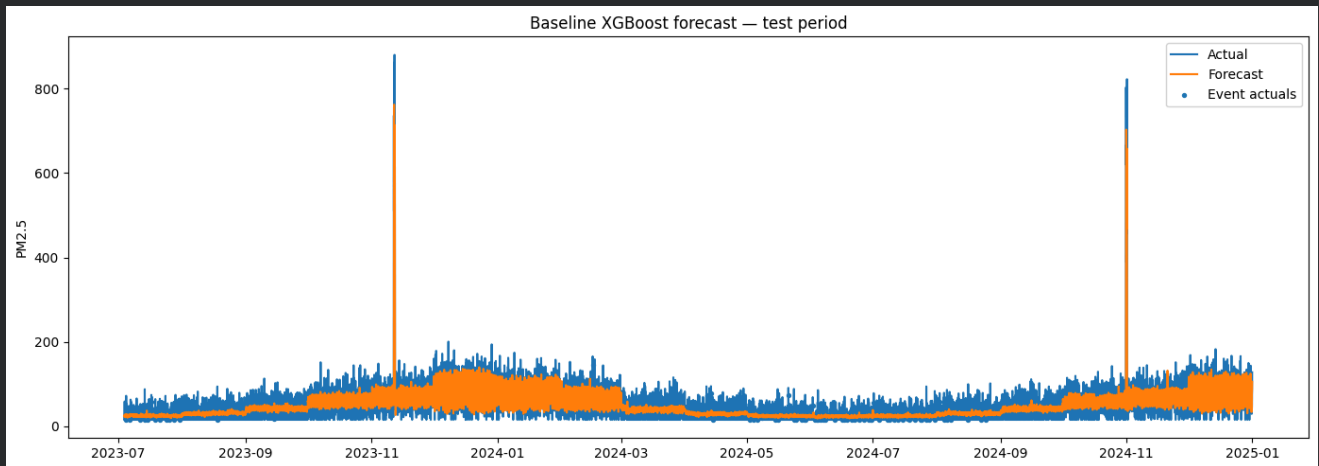
eval_df = test[["target_next", "event_flag"]].copy()
eval_df["prediction"] = test_pred
eval_df["regime"] = np.where(eval_df["event_flag"] == 1, "Event", "Normal")

rows = []
for regime, group in eval_df.groupby("regime"):
    m = regression_metrics(group["target_next"], group["prediction"])
    m["Regime"] = regime
    m["N"] = len(group)
    rows.append(m)

normal_event_results = pd.DataFrame(rows).set_index("Regime")
display(normal_event_results)

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(eval_df.index, eval_df["target_next"], label="Actual")
ax.plot(eval_df.index, eval_df["prediction"], label="Forecast")
event_idx = eval_df.index[eval_df["event_flag"] == 1]
ax.scatter(event_idx, eval_df.loc[event_idx, "target_next"], s=8, label="Event actuals")
ax.set_title("Baseline XGBoost forecast — test period")
ax.set_ylabel("PM2.5")
ax.legend()
plt.tight_layout()
plt.show()
```

	MAE	RMSE	MAPE_%	N
Regime				
Event	9.802128	13.312212	38.222781	128
Normal	16.180398	23.264637	46.115066	12998



8. Requirement I — Real CPCB sensor-data validation

Put a real CPCB observation file at:

`data/cpcb_kolkata.csv`

Recommended minimum columns:

- `datetime` (or `date` / `timestamp` / `time`)
- `pm25` (or a clearly renamed PM2.5 column)

Optional columns such as station name, PM10, NO2, SO2, CO, O3 and meteorological variables can also be retained.

The notebook never creates fake CPCB values. If the file is missing, this section reports that real-data validation is pending.

```
# =====
# 8. CPCB VALIDATION
# =====

cpcb = load_cpcb_csv(CPCB_PATH)

if cpcb is None:
    print("CPCB validation is READY but requires a real CPCB CSV at:", CPCB_PATH)
    print("No CPCB performance numbers are fabricated.")
else:
    # Normalize common PM2.5 names.
    pm_col = find_column(cpcb, ["pm25", "pm2.5", "pm_2_5", "pm2_5"])
    if pm_col is None:
        raise ValueError("CPCB file does not contain a recognizable PM2.5 column.")
    cpcb = cpcb.rename(columns={pm_col: TARGET}).copy()
    cpcb[TARGET] = pd.to_numeric(cpcb[TARGET], errors="coerce")
    cpcb = cpcb.dropna(subset=[TARGET]).copy()

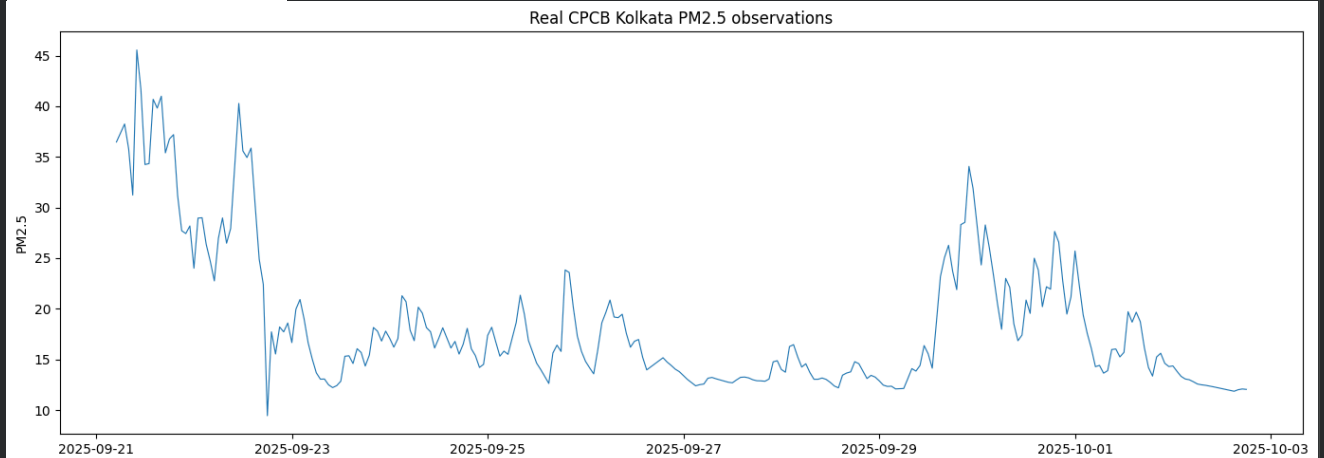
    # Resample to hourly if data are more frequent; mean preserves the measurement interpretation.
    cpcb_hourly = (
        cpcb.set_index(find_column(cpcb, ["datetime", "date", "timestamp", "time"]))
        .sort_index()[TARGET]
        .resample("1h").mean()
        .dropna()
        .to_frame()
    )

    print("CPCB hourly observations:", len(cpcb_hourly))
    display(cpcb_hourly.head())
```

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(cpcb_hourly.index, cpcb_hourly[TARGET], linewidth=0.8)
ax.set_title("Real CPCB Kolkata PM2.5 observations")
ax.set_ylabel("PM2.5")
plt.tight_layout()
plt.show()
```

CPCB hourly observations: 271

datetime	pm25
2025-09-21 05:00:00	36.4700
2025-09-21 07:00:00	38.2475
2025-09-21 08:00:00	35.7850
2025-09-21 09:00:00	31.2300
2025-09-21 10:00:00	45.5575



```
# =====
# 9. CPCB LIKE-FOR-LIKE FORECASTING VALIDATION
# =====

def prepare_single_series_for_forecasting(df, target=TARGET):
    tmp = df.copy()
    tmp["event_flag"] = 0
    tmp["event_score"] = 0.0
    return make_features(tmp, target=target)

if cpcb is not None:
    cpcb_features = prepare_single_series_for_forecasting(cpcb_hourly)
    c_train, c_val, c_test = chronological_split(cpcb_features)

    c_feature_cols = [c for c in c_train.columns if c not in {"target_next", TARGET}]
    c_model = make_regressor()
    c_model.fit(c_train[c_feature_cols], c_train["target_next"])
    c_pred = c_model.predict(c_test[c_feature_cols])

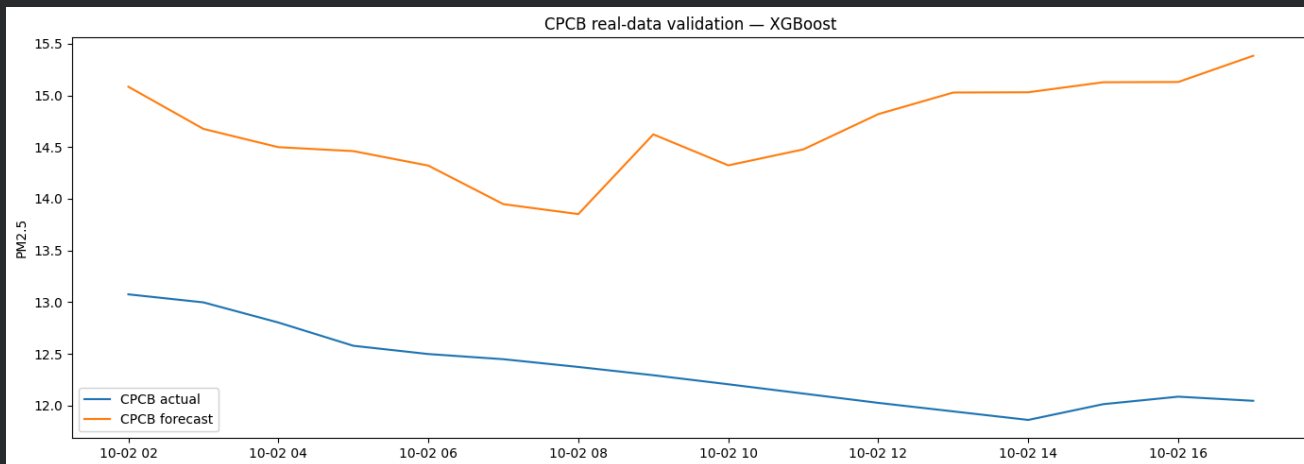
    cpcb_metrics = regression_metrics(c_test["target_next"], c_pred)
    comparison = pd.DataFrame([
        {"Dataset": "Synthetic/calibrated", **baseline_metrics},
        {"Dataset": "Real CPCB", **cpcb_metrics},
    ]).set_index("Dataset")

    display(comparison)

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(c_test.index, c_test["target_next"], label="CPCB actual")
ax.plot(c_test.index, c_pred, label="CPCB forecast")
ax.set_title("CPCB real-data validation — XGBoost")
ax.set_ylabel("PM2.5")
ax.legend()
```

```
plt.tight_layout()
plt.show()
```

	MAE	RMSE	MAPE_%
Dataset			
Synthetic/calibrated	16.118200	23.188218	46.038104
Real CPCB	2.339312	2.424304	19.097403



10. Requirement II — Event-aware adaptive forecasting

The adaptive strategy trains:

- **Normal model:** observations labelled as normal.
- **Event model:** observations labelled as extreme-event observations.

At inference time the detected regime selects the appropriate model.

This is deliberately evaluated against the ordinary single-model baseline so the research question is measurable.

```
# =====
# 10. ADAPTIVE NORMAL / EVENT MODELS
# =====

normal_train = train[train["event_flag"] == 0]
event_train = train[train["event_flag"] == 1]

print("Normal training observations:", len(normal_train))
print("Event training observations:", len(event_train))

if len(event_train) < 50:
    print(
        "WARNING: very few event observations were detected. "
        "Consider validating/tuning the event detector before interpreting the adaptive model."
    )

normal_model = make_regressor()
event_model = make_regressor()

normal_model.fit(normal_train[feature_cols], normal_train["target_next"])

if len(event_train) >= 20:
    event_model.fit(event_train[feature_cols], event_train["target_next"])
else:
    event_model = None

adaptive_pred = np.empty(len(test))

normal_mask = test["event_flag"].to_numpy() == 0
event_mask = ~normal_mask
```

```

adaptive_pred[normal_mask] = normal_model.predict(test.loc[normal_mask, feature_cols])

if event_model is not None and event_mask.any():
    adaptive_pred[event_mask] = event_model.predict(test.loc[event_mask, feature_cols])
else:
    adaptive_pred[event_mask] = model.predict(test.loc[event_mask, feature_cols])

adaptive_metrics = regression_metrics(test["target_next"], adaptive_pred)
adaptive_comparison = pd.DataFrame([
    {"Model": "XGBoost baseline", **baseline_metrics},
    {"Model": "Event-aware adaptive XGBoost", **adaptive_metrics}
]).set_index("Model")

display(adaptive_comparison)

```

Normal training observations: 60687
 Event training observations: 565

	MAE	RMSE	MAPE_%
Model			
XGBoost baseline	16.11820	23.188218	46.038104
Event-aware adaptive XGBoost	16.13163	23.064451	46.157983

```

# =====
# 11. ADAPTIVE MODEL: NORMAL VS EVENT PERFORMANCE
# =====

adaptive_eval = test[["target_next", "event_flag"]].copy()
adaptive_eval["baseline"] = test_pred
adaptive_eval["adaptive"] = adaptive_pred

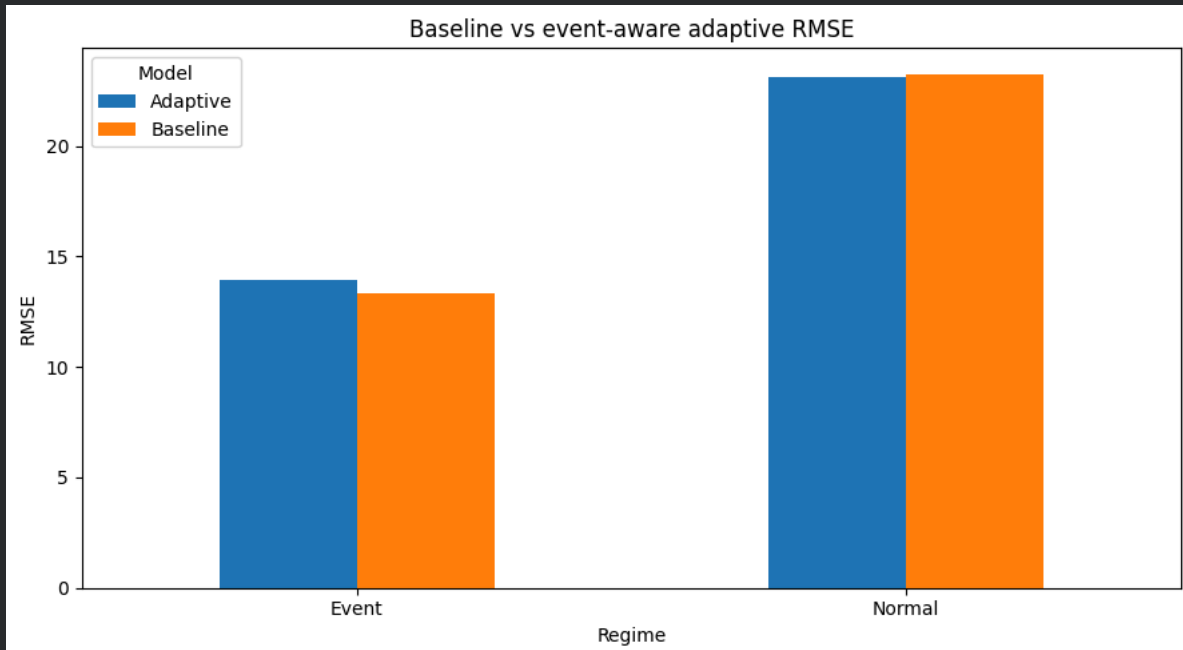
rows = []
for regime_value, regime_name in [(0, "Normal"), (1, "Event")]:
    g = adaptive_eval[adaptive_eval["event_flag"] == regime_value]
    if len(g) == 0:
        continue
    for model_name, col in [("Baseline", "baseline"), ("Adaptive", "adaptive")]:
        m = regression_metrics(g["target_next"], g[col])
        rows.append({
            "Regime": regime_name,
            "Model": model_name,
            **m,
            "N": len(g)
        })

adaptive_regime_results = pd.DataFrame(rows)
display(adaptive_regime_results)

pivot_rmse = adaptive_regime_results.pivot(index="Regime", columns="Model", values="RMSE")
ax = pivot_rmse.plot(kind="bar", figsize=(9, 5))
ax.set_title("Baseline vs event-aware adaptive RMSE")
ax.set_ylabel("RMSE")
ax.set_xlabel("Regime")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

```

	Regime	Model	MAE	RMSE	MAPE_%	N
0	Normal	Baseline	16.180398	23.264637	46.115066	12998
1	Normal	Adaptive	16.194820	23.136261	46.260922	12998
2	Event	Baseline	9.802128	13.312212	38.222781	128
3	Event	Adaptive	9.714858	13.966738	35.704903	128



12. Requirement III — Conformal prediction

We use split conformal prediction on a **calibration set** that is kept separate from the final test set.

For a point forecast $\hat{y}$, the interval is:

$$[\hat{y} - q, \hat{y} + q]$$

where q is obtained from the empirical distribution of absolute calibration residuals.

The interval is data-driven and does not assume Gaussian forecast errors.

```
# =====
# 12. SPLIT CONFORMAL PREDICTION
# =====
# Use the validation set as the calibration set.
cal_pred = model.predict(val[feature_cols])
cal_abs_resid = np.abs(val["target_next"].to_numpy() - cal_pred)

# Finite-sample conformal quantile.
n_cal = len(cal_abs_resid)
q_level = np.ceil((n_cal + 1) * (1 - ALPHA)) / n_cal
q_level = min(1.0, q_level)
q_hat = np.quantile(cal_abs_resid, q_level, method="higher")

lower = test_pred - q_hat
upper = test_pred + q_hat

conformal_df = pd.DataFrame({
    "actual": test["target_next"].to_numpy(),
    "forecast": test_pred,
    "lower": lower,
    "upper": upper,
    "event_flag": test["event_flag"].to_numpy()
}, index=test.index)

coverage = np.mean(
    (conformal_df["actual"] >= conformal_df["lower"]) &
```

```
(conformal_df["actual"] <= conformal_df["upper"])
)
mean_width = np.mean(conformal_df["upper"] - conformal_df["lower"])

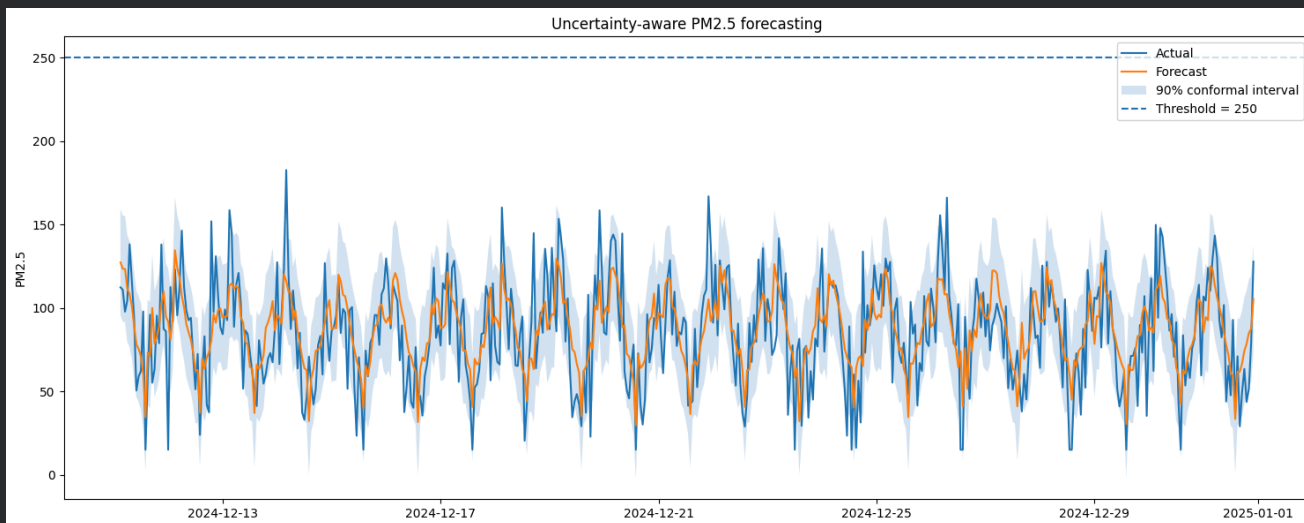
print("Conformal residual quantile:", round(float(q_hat), 3))
print("Target coverage:", f" {1-ALPHA:.1%}")
print("Observed test coverage:", f" {coverage:.1%}")
print("Mean interval width:", round(float(mean_width), 3))
```

```
Conformal residual quantile: 31.723
Target coverage: 90.0%
Observed test coverage: 88.6%
Mean interval width: 63.446
```

```
# =====
# 13. CONFORMAL FORECAST VISUALIZATION
# =====

plot_df = conformal_df.tail(min(500, len(conformal_df)))

fig, ax = plt.subplots(figsize=(15, 6))
ax.plot(plot_df.index, plot_df["actual"], label="Actual")
ax.plot(plot_df.index, plot_df["forecast"], label="Forecast")
ax.fill_between(
    plot_df.index,
    plot_df["lower"],
    plot_df["upper"],
    alpha=0.20,
    label=f" {1-ALPHA:.0%} conformal interval"
)
ax.axhline(THRESHOLD, linestyle="--", label=f"Threshold = {THRESHOLD:g}")
ax.set_title("Uncertainty-aware PM2.5 forecasting")
ax.set_ylabel("PM2.5")
ax.legend()
plt.tight_layout()
plt.show()
```



✓ 14. Pollution-threshold exceedance risk

A conservative operational risk flag is produced when the conformal interval reaches or exceeds the selected pollution threshold.

For a production deployment, a richer probabilistic calibration method can be added. For the capstone, the notebook explicitly distinguishes:

- Point forecast
- Prediction interval
- Threshold risk flag

so the uncertainty output is not confused with a deterministic forecast.

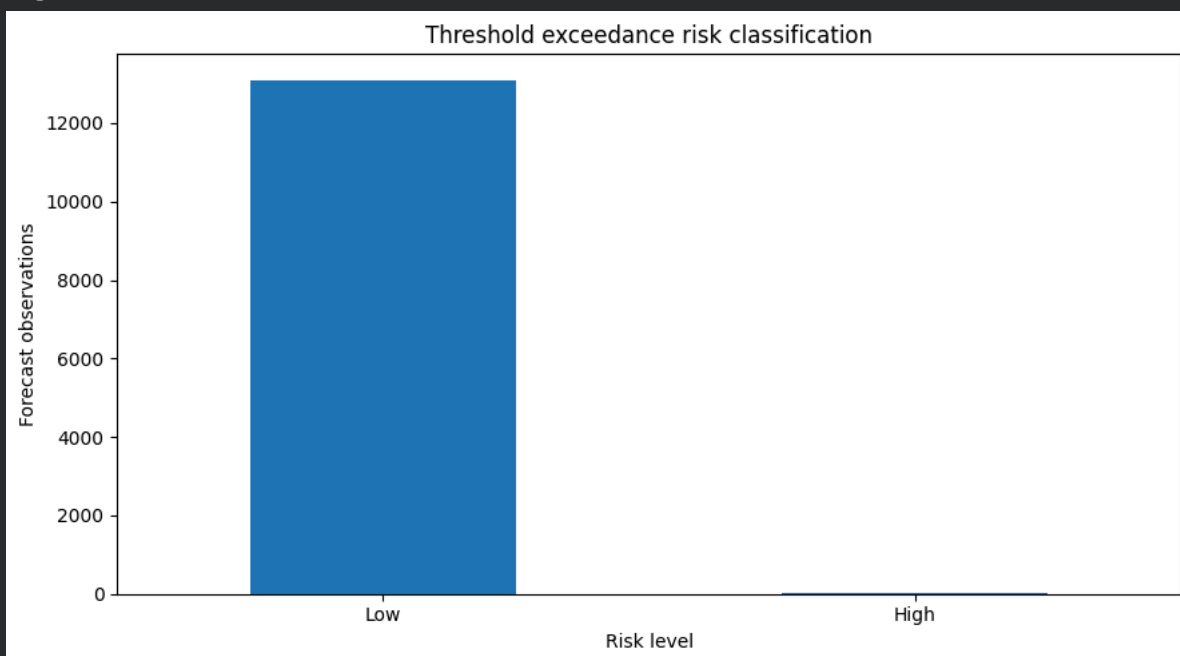
```
# =====
# 14. THRESHOLD EXCEEDANCE RISK
# =====

conformal_df["threshold"] = THRESHOLD
conformal_df["upper_exceeds_threshold"] = conformal_df["upper"] >= THRESHOLD
conformal_df["point_exceeds_threshold"] = conformal_df["forecast"] >= THRESHOLD
conformal_df["risk_level"] = np.select(
    [
        conformal_df["point_exceeds_threshold"],
        conformal_df["upper_exceeds_threshold"]
    ],
    [
        "High",
        "Watch"
    ],
    default="Low"
)

risk_summary = conformal_df["risk_level"].value_counts().rename_axis("Risk").to_frame("Count")
display(risk_summary)

fig, ax = plt.subplots(figsize=(9, 5))
risk_summary["Count"].plot(kind="bar", ax=ax)
ax.set_title("Threshold exceedance risk classification")
ax.set_ylabel("Forecast observations")
ax.set_xlabel("Risk level")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Risk	Count
Low	13102
High	24



```
# =====
# 15. COMBINED FINAL RESULTS TABLE
# =====

final_summary = pd.DataFrame([
    {
        "Experiment": "XGBoost baseline",
        "MAE": baseline_metrics["MAE"],
    }
])
```

```

"RMSE": baseline_metrics["RMSE"],
"MAPE_%": baseline_metrics["MAPE_%"]
},
{
"Experiment": "Event-aware adaptive XGBoost",
"MAE": adaptive_metrics["MAE"],
"RMSE": adaptive_metrics["RMSE"],
"MAPE_%": adaptive_metrics["MAPE_%"]
}
])

display(final_summary)

uncertainty_summary = pd.DataFrame([
    "Target coverage": 1 - ALPHA,
    "Observed coverage": coverage,
    "Mean interval width": mean_width,
    "Threshold": THRESHOLD,
    "Watch/High fraction": np.mean(conformal_df["risk_level"] != "Low")
])

display(uncertainty_summary)

```

	Experiment	MAE	RMSE	MAPE_%	
0	XGBoost baseline	16.11820	23.188218	46.038104	
1	Event-aware adaptive XGBoost	16.13163	23.064451	46.157983	
	Target coverage	Observed coverage	Mean interval width	Threshold	Watch/High fraction
0	0.9	0.886409	63.44628	250.0	0.001828

16. Supervisor presentation visualizations

Recommended figures for the final report/presentation:

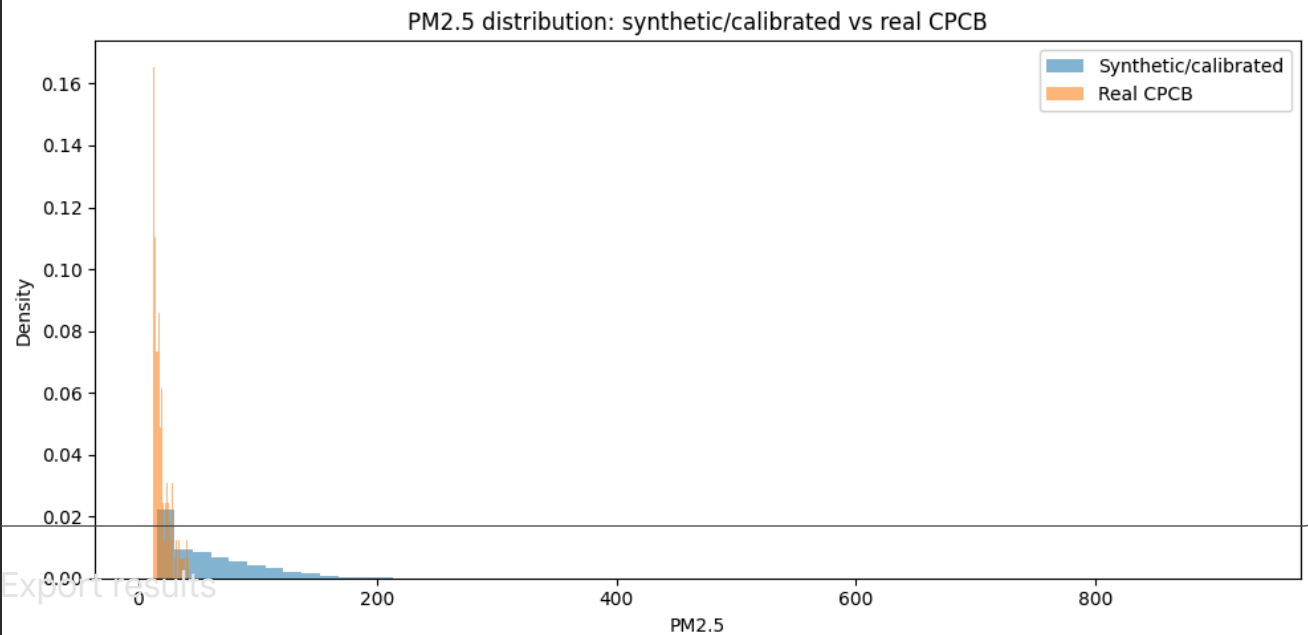
1. Synthetic vs real CPCB PM2.5 distributions.
2. Synthetic vs CPCB forecasting error table.
3. Normal vs event RMSE.
4. Baseline vs adaptive event-model RMSE.
5. Actual vs point forecast with conformal interval.
6. Threshold and exceedance-risk timeline.
7. Event-detection timeline.
8. A final dashboard combining current regime, forecast, interval and risk.

The notebook above creates the core figures without fabricating CPCB results.

```

# =====
# 16. OPTIONAL SYNTHETIC VS CPCB DISTRIBUTION
# =====
if cpcb is not None:
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.hist(synthetic[TARGET].dropna(), bins=60, alpha=0.55, density=True, label="Synthetic/calibrated")
    ax.hist(cpcb_hourly[TARGET].dropna(), bins=60, alpha=0.55, density=True, label="Real CPCB")
    ax.set_title("PM2.5 distribution: synthetic/calibrated vs real CPCB")
    ax.set_xlabel("PM2.5")
    ax.set_ylabel("Density")
    ax.legend()
    plt.tight_layout()
    plt.show()
else:
    print("Distribution comparison will appear automatically after CPCB data are supplied.")

```



17. Export results

The following cell saves the principal tables so they can be inserted into the report.

```
# =====
# 17. EXPORT RESULTS
# =====

RESULTS_DIR = Path("./results")
RESULTS_DIR.mkdir(exist_ok=True)

normal_event_results.to_csv(RESULTS_DIR / "normal_vs_event_baseline.csv")
adaptive_comparison.to_csv(RESULTS_DIR / "baseline_vs_adaptive.csv")
adaptive_regime_results.to_csv(RESULTS_DIR / "adaptive_regime_results.csv")
conformal_df.to_csv(RESULTS_DIR / "conformal_forecasts.csv")
risk_summary.to_csv(RESULTS_DIR / "threshold_risk_summary.csv")

if cpcb is not None:
    comparison.to_csv(RESULTS_DIR / "synthetic_vs_cpcb.csv")

print("Results exported to:", RESULTS_DIR.resolve())
```

Results exported to: M:\capston_project_2\project\results

18. Research interpretation checklist

Before presenting to the supervisor, report the following: